Investor & Partner Pitch

SecureAIExam

A leak-proof, end-to-end encrypted examination platform —
from question bank to sealed answer sheet, every step signed, sealed, and witnessed.

Prepared as a technical & business pitch · Confidential

1. The Problem

A single leaked question paper can force the cancellation of an exam that millions of candidates spent months preparing for. It doesn't take a sophisticated attack — it takes one weak link in a process that, today, has no way to prove where it broke.

Why leaks keep happening

- **No custody trail.** Once a paper is printed, it moves through transport, storage, and handling with no record of who held it or when. A missing copy is only noticed when it's too late.
- **No way to trace a leak back to its source.** A single leaked photo tells investigators nothing about which physical copy it came from, which center it was meant for, or who had access to it. Every leak becomes an unsolved case.
- **No real-time visibility.** Problems are discovered after the exam has already been compromised, when the only remaining option is a costly, disruptive re-exam.
- **Security is bolted on, not built in.** Locked boxes, sealed envelopes and manual sign-off sheets rely entirely on the honesty of every person who touches the paper — there is no cryptographic guarantee behind any of it.

The core failure: today's process depends on trusting every individual in the chain. SecureAIExam is built on the opposite principle — trust no single individual, and make every step provably verifiable instead.

2. The Solution, at a Glance

SecureAIExam replaces "trust the process" with "verify the process." Four pillars work together across the paper's entire lifecycle:

Pillar 1 — Dual-Key Encryption

A paper is encrypted the moment it's composed. The Paper Setter and the Admin each hold one half of the key. Neither can decrypt the paper alone — both passphrases must be entered together, at the print ceremony, for the paper to ever become readable.

Pillar 2 — Signed Physical Custody

Every printed copy carries a cryptographically signed QR code from the moment it comes off the press. A forged or duplicated code fails verification instantly when scanned — it is mathematically impossible to fake a valid one without the server's private signing key.

Pillar 3 — Forensic Tracing

Beyond the header QR, every individual question on every copy carries its own invisible, signed watermark. Even a single photographed question — not the whole paper — can be traced back to the exact copy and page it was printed on.

Pillar 4 — Live Oversight

Every seal, scan, and anomaly is written to a hash-chained, tamper-evident audit log and pushed to the Admin's dashboard in real time. Editing a past record breaks every record that follows it, making silent tampering detectable by construction.

3. Who Uses the Platform

Four distinct roles, each with access scoped to exactly what their job requires — enforced at the database level, not just in the app's UI.

Role	Responsibility	What they can never do
Admin	Schedules exams against a certified format, co-signs papers (key share 2 of 2), runs the print ceremony, provisions exam centers, and monitors the live audit and alert feed.	Cannot decrypt or print a paper without the Setter's passphrase as well.
Teacher	Submits questions to the bank — by hand or via PDF import with automatic NLP extraction — within a subject and quota set by the exam format.	Cannot see or influence how the final paper is assembled.
Paper Setter	Assembles each paper set from the bank, previews the exact final rendered paper, and seals it with their own passphrase (key share 1 of 2).	Cannot decrypt or print a paper without the Admin's co-signature.
Invigilator / Center	Signs in with a center-specific login, scans every copy's QR at the exam hall, and clears it from the pending list in real time.	Cannot scan in a copy that was not allocated to their specific center.

4. The End-to-End Journey

This is the exact path a real exam takes through the platform, from the moment it's scheduled to the moment its printed copies are accounted for at the exam hall.

Step 1 — Exam scheduled. The Admin selects a certified exam format (e.g. a national entrance exam or board pattern) and a year. The platform loads that format's full blueprint automatically — sections, subjects, question counts, marking scheme, negative marking, and duration — so nobody can compose a paper that breaks the official pattern, even by accident. The Admin sets how many independent paper sets are needed (Set A, Set B, …) and whether questions may repeat across sets.

Step 2 — Questions are banked. Teachers submit questions by hand, or upload a PDF that an NLP pipeline parses into structured questions automatically. Every submission is checked against the exam format's rules for that section and capped at twice what the exam actually needs — enough headroom for the Setter to choose between options, not so much that the bank grows unbounded.

Step 3 — A paper is composed. The Paper Setter picks a set, selects questions from the bank until every section's quota is satisfied, and opens a full preview of the exact final document — header, instructions, sections, continuous numbering, marks and negative marking, rendered exactly as it will be printed. Nothing can be sealed until this preview has been explicitly confirmed.

Step 4 — Sealed (key share 1 of 2). The Setter enters their own passphrase. The server generates a random 256-bit encryption key, encrypts the confirmed paper with it, and splits that key into two shares using an XOR split — a scheme where a single share reveals mathematically zero information about the original key. The Setter's share is wrapped with their passphrase; the second share is held in escrow until co-signing.

Step 5 — Co-signed (key share 2 of 2). The Admin enters their own passphrase. The escrowed second share is now wrapped with it, and the escrow copy is destroyed. From this exact moment, decrypting the paper requires both passphrases entered together — no single person, including a compromised Admin or a compromised Setter account, can ever decrypt it alone.

Step 6 — The print ceremony. Both passphrases are entered together, once. The paper is decrypted in memory, rendered into its final printable form, and for every physical copy requested: a unique, cryptographically signed QR code is generated for its header, and an invisible signed watermark is embedded beside every individual question. Neither the QR nor the watermark can be produced without the server's private signing key — a forged copy fails verification the instant it's scanned.

Step 7 — Dispatch and allocation. Printed copies are allocated to specific exam centers in the database. Each copy's QR is now tied to that exact center — scanning it anywhere else, or with any other center's invigilator login, is rejected.

Step 8 — Scan-in at the center. The invigilator signs in with a center-specific login and scans every copy's QR with an ordinary phone camera. Each scan verifies the signature, marks that copy as received, and instantly notifies the Admin's dashboard. If any copy remains unscanned as the exam's

start time approaches, a critical alert fires automatically — there is no scenario where a missing copy goes unnoticed until it's too late.

Step 9 — Custody, always tracked. From printing press to exam hall, every checkpoint a copy passes through is written as one more link in a hash-chained record tied to that specific copy. If a copy is ever reported missing or a leak is identified, its entire history — who held it, where, and when — is available instantly.

Step 10 — The audit trail. Every seal, co-sign, print, scan and alert — across every paper, every copy, every center — is written to one global, hash-chained, tamper-evident log. Altering any single past entry breaks every entry that follows it, so tampering isn't just against policy — it's mathematically detectable.

5. Deep Dive — Encryption & QR Security

Dual-control sealing

The scheme guarantees that decryption is impossible without both the Setter's and the Admin's passphrases, entered together, at the same moment:

```
K = 32 random bytes (the actual AES-256-GCM paper key) shareA = 32 random bytes
(Setter's share) shareB = K XOR shareA (Admin's share, derived, not stored as K)
wrap(shareA) with Setter passphrase -> scrypt (N=16384) -> AES-256-GCM wrap(shareB)
with Admin passphrase -> scrypt (N=16384) -> AES-256-GCM [after co-sign]
decrypt(paper) requires: unwrap(shareA) XOR unwrap(shareB) == K
```

Because `shareA XOR shareB = K`, possessing only one share reveals **zero bits** of information about K — this is information-theoretically secure secret splitting, not merely "hard to guess."

The signed QR payload

Every printed copy's header QR encodes a small JSON payload, signed by the server:

```
{ "copy_id": "<uuid of this physical copy>", "paper_id": "<uuid of the sealed
paper>", "copy_number": 47, "sig": HMAC-SHA256(server_secret, copy_id + paper_id +
copy_number) }
```

On every scan, the server recomputes the signature from the payload's own fields and compares it to `sig`. Because the signature depends on a secret the server never exposes, a forged, edited, or reused QR fails verification immediately — there is no way to hand-craft a valid one without the server's key.

Per-question forensic watermarking

Alongside the header QR, every individual question carries its own short, signed watermark token encoding the copy, the page, and the question number. If a leak surfaces as a single photographed question — not the full paper — that token alone is enough to name the exact copy and question it came from, without needing the rest of the page.

Tamper-evident hashing

Both the global audit log and each copy's custody trail use the same chaining construction:

```
entry.hash = SHA-256( prev_entry.hash || actor || action || entity || details ||
timestamp )
```

Every entry's hash depends on the one before it. Rewriting any historical entry requires recomputing every hash after it — and the log is append-only at the database level, so that recomputation has nowhere valid to be written.

6. Database Structure

The schema is organized around one idea: nothing sensitive is ever readable by a client directly — every table is governed by row-level security scoped to the requester's role, and the one table holding encrypted key material has *no client-facing policy at all.*

Identity & access

Table	Purpose
`profiles`	One row per account — role (Admin / Teacher / Paper Setter / Invigilator), and an approval status gating access until an Admin approves the sign-up.
`teacher_subjects`	Which subjects a teacher may submit questions for, per exam.
`setter_domains`	Which exam categories a Paper Setter is authorized to compose papers for, account-wide.

Exam definition

Table	Purpose
`exams`	One row per scheduled exam — its certified format, year, date, how many paper sets it needs, and whether questions may repeat across sets.
`exam_slots`	The day's timing structure — 2–3 dynamic slots with their own start, end and duration.
`exam_centers`	Each physical exam center, its provisioned invigilator login, and which slot it runs.

Question bank & papers

Table	Purpose
`questions`	The bank itself — body, type (MCQ / MSQ / numerical / theoretical), marks, answer key, and which exam-format section it fills. A check constraint enforces that every question's shape matches its declared type.

`papers`	One row per (exam, set) — its lifecycle status (draft → sealed → dual-sealed → printed) and the timestamps of each ceremony.
`paper_questions`	Which questions belong to which paper, and in what order.
`paper_secrets`	Ciphertext and the wrapped key shares only. **Zero row-level security policies exist for this table by design** — no client credential, of any role, can ever read it. Only the backend service, using its own privileged connection, touches it.

Custody & forensics

Table	Purpose
`paper_copies`	One row per physical printed copy — its signed QR payload, trace code, fingerprint hash, current status, allocated center, and scan-in timestamp.
`custody_events`	The hash-chained history of one copy — every checkpoint it passed through, each entry linked to the one before it.
`qr_detections`	Per-question watermark sightings — which copy, which page, which question, and where the detecting device was.

Oversight

Table	Purpose
`alerts`	Real-time notifications — unscanned copies approaching a slot's start, a copy reported missing, a leak identified.
`audit_log`	The global, hash-chained, append-only record of every seal, co-sign, print, scan and alert across the entire platform.

7. Technology Stack

Layer	Technology	Why
Mobile & Web App	React Native, Expo	One codebase serves every role, on phone and browser alike — no separate web and mobile builds to maintain.
Crypto Service	Node.js, Express	Owns the exam-format catalogue and enforces it server-side — sealing, co-signing, printing and forensics all pass through here, so a client can never write a paper that breaks the rules, even by calling the database directly.
AI Service	Python, FastAPI	NLP extraction of questions from uploaded PDFs, per-copy fingerprinting, and custody anomaly scoring — stateless, so it scales horizontally behind a load balancer.
Database	Postgres (Supabase)	Row-level security enforced per role at the database itself, plus realtime subscriptions that push updates to dashboards the instant a row changes.
Monitoring	Prometheus, Grafana	Request health and business metrics — papers sealed, copies scanned, critical alerts fired — on one always-on dashboard.

8. Threat → Mitigation Map

Threat	Mitigation
An insider — Setter, Admin, or printer — opens the paper early.	Dual-control key split. No single passphrase decrypts anything.
The database itself is breached.	The secrets table holds only ciphertext and passphrase-wrapped shares, with no client-facing read policy at all — a stolen database dump is still unreadable without the passphrases.
A physical copy goes missing in transit.	Per-copy custody chain. A missed checkpoint raises an instant critical alert naming the last known holder.
A photographed page leaks on social media.	Invisible per-question fingerprint → exact copy → exact custody trail → exact person who last held it.
Someone attempts to forge an "official" copy.	QR payloads are HMAC-signed server-side. Verification fails instantly for anything not signed with the server's own key.
Logs are edited after the fact to hide involvement.	Hash-chained, append-only audit log. Editing one entry breaks every hash computed after it.
The client app itself is tampered with.	Every authorization check runs server-side — row-level security plus role checks on every request. The app is treated as untrusted by design.

9. Built for Exam Day, Not Just a Demo

- **Custody scans** are single-row inserts — a write pattern Postgres already handles at tens of thousands per minute, with custody events partitioned by exam for clean, fast archives.
- **Realtime fan-out** multiplexes every dashboard's WebSocket connection, and each dashboard subscribes only to the filtered channel it actually needs — not a firehose of every event on the platform.
- **The AI service** is fully stateless, so it scales horizontally behind a load balancer; the expensive fingerprinting step happens once, at print time, never on the live request path.
- **The crypto service** is stateless aside from its database connection. Sealing, co-signing and printing are rare ceremonies — once per paper, never once per copy — so the service that guards the most sensitive operation is also the one under the least continuous load.

10. Roadmap

What ships next, building directly on the architecture already in place:

- **Computer-Based Testing (CBT) mode.** Extend the same dual-key, custody-tracked pipeline to fully digital delivery — encrypted papers rendered directly on secure center terminals, unlocked by the same seal / co-sign ceremony, with no printed copy to leak at all.
- **Higher-accuracy camera scan-in.** A computer-vision model that reads a copy's identity and verifies its watermark from an ordinary phone photo — even under glare, tilt, or a damaged corner — so scan-in stops depending on a perfectly clean, aligned shot.
- **Cloud-native, containerized scaling.** Package the crypto and AI services as Docker images behind an orchestrator, so exam-day load scales out horizontally on demand and back down again once the exam ends.
- **Multi-language papers from one composition.** Generate and seal the same paper in every language a board offers from a single composed set, instead of a parallel manual process per language.
- **Offline-first center sync.** Let a center scan and queue custody events without connectivity, syncing the moment a network is reachable — built for rural and low-bandwidth exam halls.
- **Publicly verifiable audit trail.** Periodically anchor the hash-chain to a public ledger, so the tamper-evidence claim doesn't depend on trusting the platform's own database.

Demo-to-production honesty: today's build makes a few deliberate simplifications for demo purposes — a brief server-side escrow window between seal and co-sign (production: hardware-backed key ceremony), simpler steganographic watermarking (production: frequency-domain watermarking that survives a print-and-photograph round trip), and self-selected roles at sign-up (production: invite-only provisioning by the examining board). None of these change the architecture — they're hardening steps on top of it.

11. The Ask

SecureAIExam is a complete, working system today — not a concept. Every flow described in this document, from scheduling an exam to scanning a QR at the exam hall, runs end-to-end in the current build.

What we're looking for: a pilot partner — one exam board, one exam cycle — to prove this architecture at real scale. The system is ready to run live; we want the chance to.

Built so no single person — and no single leaked page — can bring an exam down.